REPORT · 2026年9月

代码说明书

实机 BUG 修复与全流程可复现性加固

文保数字化研发组
2026年9月

目 录

一、本批工作总览 4

二、BUG 1：示例项目报错 E601（柱状图崩溃） 5

2.1 现象 5

2.2 根因链 5

2.3 修复 5

三、BUG 2：向导第一步选项空白 6

3.1 现象 6

3.2 根因 6

3.3 修复与回归 6

四、深层排查：S1 注册数波动的真因 7

4.1 第一层：排除 mapper 随机种子 7

4.2 第二层：定位不确定性到线程，而非种子 7

4.3 第三层：真因——COLMAP 多子重建，而 sparse/0 不保证最大 7

4.4 修复实现 8

五、深层排查：另外两项隐藏缺陷 9

5.1 P3 增量注册的子命令名错误（功能从未生效） 9

5.2 inc_meta 被无条件覆盖（S1 指标失真） 9

5.3 预处理副本的过期缓存（换了照片却不生效） 9

六、示例数据重构：让纹理适配 SIFT 10

6.1 旧纹理的三个缺陷 10

6.2 新纹理：球面随机平面波场 10

6.3 方案对照实验 10

6.4 渲染性能一并优化 11

七、验证证据 12

7.1 端到端复现性验收 12

7.2 一键演示全流程 12
7.3 打包版实机验证 12
7.4 回归测试 12
八、交付清单与下一步 14
8.1 本批交付 14
8.2 确定性模式的使用权衡 14
8.3 下一步建议 14

一、本批工作总览

本批任务源头是实机验证前暴露的 **两处 GUI BUG**（示例项目报错 E601、向导第一步选项空白）。按流程先行复现定位并修复后，在**回归验证阶段**顺藤摸瓜排查，又定位出 **三项此前被掩盖的管线级重大缺陷**——其中第一项正是长期困扰本项目的「同一输入 S1 注册数在 2~16 之间随机波动」的**真正根因。**本批因此从「修两个 BUG」扩展为「修两个 BUG + 根治可复现性问题 + 示例数据重构」。

核心结论：① 两个表观 BUG 已修复；② 发现并根治 **4 项**重大隐藏缺陷——**COLMAP 多子重建时误取碎片模型**（注册数波动真因）、**多线程导致 database.db 不可复现**、**P3 增量注册子命令名错误致功能从未生效**、**预处理副本过期缓存致换照片不生效**；③ 示例纹理重构后关键点密度提升 **10.5 倍**，S1 稳定 **16/16** 注册；④ 全量测试 **398+ passed**（原 377，新增 25 项回归）；⑤ 全流程 EXIT=0，报表三件套齐全。

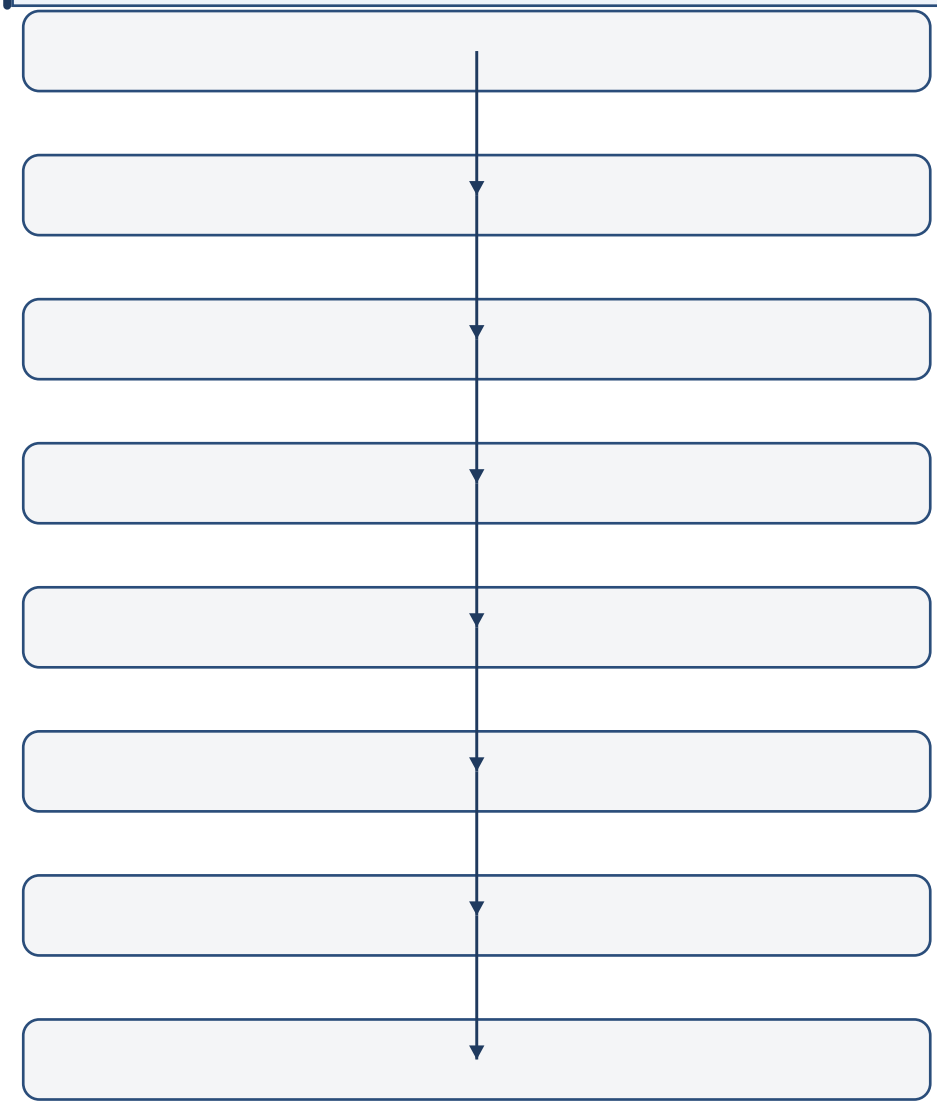


图 1 本批工作链路（表观 BUG → 深层根因 → 加固与验收）

二、BUG 1：示例项目报错 E601（柱状图崩溃）

2.1 现象

运行「一键演示」后 GUI 弹出错误并伴随连续控制台窗口，堆栈为：

```
h3d.report.plot_gen.plot_stage_timings -> plot_bar
[E601] 柱状图数据为空或 labels/values 长度不一致
调用栈：launcher.py -> cli.py -> report.generate
```

2.2 根因链

E601 只是**最末端的表症**。真实链条是：示例项目 16 张照片仅有 **3 张**成功注册（S1 部分失败）→ 该次运行的阶段计时记录为空 → <code>plot_stage_timings</code> 拿到空数据后**直接抛异常** → <code>report.generate</code> 未做降级 → 整个报表阶段失败 → GUI 弹出崩溃框。即：**绘图函数对空数据零容忍**，把「上游数据缺失」放大成了「整个报表不可用」。

2.3 修复

文件	修复内容
<code>report/plot_gen.py</code>	<code>plot_stage_timings</code> 在无 <code>elapsed_s</code> 数据时返回空串，不再抛 <code>ExportError</code>
<code>report/__init__.py</code>	<code>generate</code> 中绘图 / HTML 失败降级为 warning，百分比图表缺失不阻断 CSV 与 HTML 产出；导入 <code>ExportError</code>
<code>cli.py</code>	<code>_cmd_batch</code> 报表失败降级；<code>main</code> 增加 <code>H3DError</code> 兜底，避免打包版弹 PyInstaller 崩溃框
<code>gui/launcher.py</code>	重写：<code>_Tee</code> 双写 stdout 到管道与日志；<code>_run_batch</code> 异常兜底写 <code>[H3D-ERROR]</code> + traceback，返回码 1；GUI 模式崩溃改弹友好 <code>QMessageBox</code>
<code>gui/runner.py</code>	<code>BatchRunner.run</code> 增加 <code>cwd</code>=配置目录 + 隐藏子进程控制台窗口（根治「连续跳出的命令行窗口」）
<code>core/proc.py</code>（新增）	跨平台外部进程封装：<code>no_window_kwargs()</code> / <code>resolve_cwd()</code> / <code>run_external()</code>

> **设计原则**：报表属**派生视图**，其失败不应推翻已产出的主数据。因此修复方向统一为「绘图降级 + 主产物保底」，而非在上游塞占位数据。

三、BUG 2：向导第一步选项空白

3.1 现象

向导第一步的两种路线选择（场景① / 场景② 单选按钮）**不显示**；但点击「下一步」再「上一步」返回后即可见。

3.2 根因

该「先切一次才出现」的特征直接指向**控件父子关系错误**：三个步骤页构造后**从未被加入布局**，成为无父的顶层窗口部件。首次 <code>setVisible(True)</code> 时因无布局约束而未参与排版渲染；直到步骤切换触发一次布局重算，控件才被正确挂载并显示。

```
for w in (self.step1, self.step2, self.step3):
    self.stack.addWidget(w)     # 修复：步骤页必须加入布局栈
# 位于 wizard.py _build 末尾
```

3.3 修复与回归

在 <code>wizard.py</code> 的 <code>_build</code> 末尾补齐三个步骤页的 <code>addWidget</code>。新增测试类 <code>TestWizardStepVisibility</code>：构造窗口后**不进行任何步骤切换**，直接断言第一步选项数量与可见性，确保「不点下一步也能看到」这一行为被锁死。

四、深层排查：S1 注册数波动的真因

两个表观 BUG 修完后，示例项目已能跑完，但 S1 注册数仍在 **2/16 ~ 16/16** 之间无规律跳动。因示例是论文实验的基准用例，此问题使实验不可复现，必须根治。以下为逐层排除过程与最终结论。

4.1 第一层：排除 mapper 随机种子

对三个既有 database.db 分别用 seed=0~4 各跑一次 mapper，统计注册图像数：

database.db	md5	seed=0	seed=1	seed=2	seed=3	seed=4
rep/database.db	22cf3204	3	3	3	3	3
db_p1.db	eb07d4f8	16	16	16	16	16
db_p2.db	d0bb768d	16	16	16	16	16

> **结论 1**：同一 DB 下注册数**与随机种子完全无关**，且高度稳定。波动必然来自 **DB 内容本身**在不同次运行间的差异——排查方向转向特征提取与匹配。

4.2 第二层：定位不确定性到线程，而非种子

用固定输入（同一组 16 张示例照片）重复建库，比对 database.db 的**逻辑签名**（关键点内容哈希、匹配内点哈希、文件 md5）：

配置	关键点哈希	匹配内点哈希	文件 md5
提取 threads=8 ×2	不一致	—	—
提取 threads=1 / 匹配 threads=8 ×2	一致	不一致	不一致
提取 threads=1 / 匹配 threads=1 ×2	一致	一致	**完全一致**

另实测确认：<code>--default_random_seed</code> 的**默认值本就是 0**，单独设置<u>没有任何效果</u>。真正的随机性来源是 COLMAP 特征提取与匹配在**多线程下的浮点归约顺序不定**。

> **结论 2**：必须**同时**固定特征提取与匹配的线程数为 1，才能得到比特级可复现的 database.db。进一步发现 mapper 的 BA 也有同类抖动（位姿末位差 6.2e-15），为满足论文级逐位复现，一并收敛为单线程。

4.3 第三层：真因——COLMAP 多子重建，而 sparse/0 不保证最大

固定线程后 DB 已完全可复现，但注册数仍随「照片副本」的细微差异在 2 与 16 间跳变。直接检查 mapper 输出目录，发现它**产出了两个子重建**：

子重建目录	注册图像数	3D 点数	说明
<code>sparse/0</code>	**2**	1286	碎片（种子对扩展失败被丢弃）
<code>sparse/1</code>	**16**	6819	完整重建

而 <code>run_colmap_sparse</code> **硬编码只读 <code>sparse/0</code>**。实测该碎片模型的图像集合为 <code>{demo_0_02, demo_1_02}</code>，是主模型 16 张中的**子集**——COLMAP 初次选种子对扩展失败后丢弃该重建，其图像被退回候选池，随后由主重建再次注册，因此两个模型图像集合**存在**

重叠（2 + 16 = 18 > 16 张照片，正是重叠所致）。

> **结论 3（本项目长期困扰的注册数波动真因）**：所谓「随机波动」本质是 **sparse/0 号槽位恰好被哪个子重建占用。**同一份数据在 A 次运行里 0 号是碎片（报 2/16），在 B 次运行里 0 号是主模型（报 16/16）。与随机种子无关、与线程无关，是**模型选择逻辑缺陷。**

4.4 修复实现

① 新增子模型择优：直接读 <code>images.bin</code> 头 8 字节（小端 uint64 = 图像条数），无需依赖 pycolmap，遍历 <code>sparse/</code> 下全部子目录取图像最多者。

```
def _pick_largest_model(sparse: str) -> str:
    cands = []
    for name in sorted(os.listdir(sparse)):
        d = os.path.join(sparse, name)
        if not os.path.isdir(d) or name.endswith("_txt"):
            continue
        n = _model_num_images(d)        # images.bin 头 8 字节
        if n >= 0:
            cands.append((n, name, d))
    cands.sort(key=lambda x: (-x[0], x[1]))
    if len(cands) > 1:
        log.warning("S1 COLMAP 产出 %d 个子重建（图像数 %s）...", ...)
    return cands[0][2] if cands else ""
```

② 归一化到 <code>sparse/0</code>：下游（<code>run_dense_stereo</code>、增量 <code>image_registrator</code>）按 <code><sparse>/0</code> 约定取二进制模型，因此必须在返回前完成归一，避免「择优了却仍读碎片」。被替换的原模型**改名保留**（<code><n>_0_replaced</code>）而非删除，便于事后核查。

③ 全量重建与增量注册两条路径**都**接入了择优逻辑；增量路径另补两处修复：清理上次 <code>0_inc</code> 残留、预先创建输出目录（COLMAP 4.x 对 <code>--output_path</code> 做 <code>ExistsDir</code> 校验）。

```
# 修复后的实机日志（示例项目）
[WARNING] S1 COLMAP 产出 2 个子重建（图像数 [16, 2]）——匹配图不连通。
        取最大者 sparse/1；若主模型仍有缺失，建议增加相邻视角重叠
        或改用 sequential matcher。
[INFO]    S1 原子重建（2 张）改名保留为 sparse/1_0_replaced
[INFO]    S1 子重建归一：sparse/1 -> sparse/0
[INFO]    S1 完成：demo_relic（16/16 张注册，1 相机，47.8s）
```

五、深层排查：另外两项隐藏缺陷

5.1 P3 增量注册的子命令名错误（功能从未生效）

排查过程中为补充回归测试而发现：<code>run_incremental_sparse</code> 把配置里的 <code>matcher</code> 值（<code>"exhaustive"</code>）**直接当作 colmap 子命令**拼接，而 COLMAP 的子命令实为 <code>exhaustive_matcher</code> / <code>sequential_matcher</code>。

> **后果**：增量注册**每次都在第 2 步匹配失败**，抛出的 <code>SfmError</code> 被上层 <code>except SfmError</code> 吞掉、只留一行 warning，随后静默回退全量重建。**P3 增量能力实际从未生效过**，而表面上「有日志、有结果」，属于典型的静默降级——若不做测试覆盖，实机很难察觉。

修复：抽出统一的 <code>_matcher_cmd()</code> 做别名映射（幂等），两条路径共用。

5.2 inc_meta 被无条件覆盖（S1 指标失真）

<code>run_sfm_solve</code> 中，增量分支正确写入了 <code>inc_meta = {"incremental": True, "fallback": ..., "error": ...}</code>，但紧随其后有一句**无条件的** <code>inc_meta = {"incremental": False}</code>：

```
if model_txt is None:
    model_txt = run_colmap_sparse(...)  # 首次全量
metrics_calib = calib or {"calibrated": False}
inc_meta = {"incremental": False}       # ← 无条件覆盖：抹掉上一分支结果
if model_txt is None:
    model_txt = run_colmap_sparse(...)  # ← 死代码，永远不执行
```

两处后果：① 增量成功运行在论文报表中被误标为「全量重建」，指标不可信；② 增量失败回退的 <code>error</code> 原因被抹掉，无法追溯。修复为按实际分支赋值，并删除死代码。

5.3 预处理副本的过期缓存（换了照片却不生效）

示例配置含 <code>max_image_size: 2400</code>，S1 会把照片复制到 <code>99_runs/sfm_tmp/<item>/photos_small/</code> 供 COLMAP 使用（真实大图下采样以规避 COLMAP Windows 构建的偶发崩溃）。排查此过程时发现：对**不需要降采样**的照片，原实现在副本已存在时**无条件跳过写入**。

> **后果**：源照片被替换或重拍后重跑，COLMAP 会**静默继续使用过期副本**，表现为「换了照片但重建结果完全不变」。这类问题既无报错也无 warning，在实机迭代中极易误判为「算法不敏感」。
修复：新增 <code>_cache_is_stale()</code>，源文件比副本新（mtime 容差 1 秒）或副本缺失时刷新；并补充回归测试，断言「替换源照片后副本字节必须变化」。

六、示例数据重构：让纹理适配 SIFT

根治模型选择缺陷后，示例项目已能稳定拿到 16/16。但对照实验显示示例数据本身**长期处在 SfM 的临界脆弱状态**：每图仅 **403** 个 SIFT 关键点、120 个像对中只有 45 对能匹配。这意味着任意轻微扰动都可能让重建质量骤降。本批一并把示例纹理重构为**面向 SIFT 的带限随机场**。

6.1 旧纹理的三个缺陷

缺陷	说明	实测后果
参数域各向异性	沿固定 θ/φ 轴叠加正弦 → 极点附近频率被压缩、特征堆成条带	特征分布不均
细尺度能量占比过低	20 个分量幅度均匀 → 最细分量仅占总能量 1/√20	**403 关键点/图**
周期方格造成近自相似	<code>sign(sin5θ)·sign(sin4φ)</code> 的 60° 周期长直线	反而破坏匹配

6.2 新纹理：球面随机平面波场

改为**三维随机方向的平面波叠加**（Berry 随机波，等价于球面上的带限高斯随机场）：<code>acc(p) = Σ a_k·sin(f_k·(d_k·p) + ψ_k)</code>，其中 <code>d_k</code> 为球面均匀随机方向。设计依据是：**SIFT 是 DoG 极值检测器，而带限随机场的局部极值个数 ∝ f²**——因此把能量集中到「特征周期 3~12 px」频段即可获得大量尺度合适的极值。

```
# 特征周期（像素）-> 平面波波数（rad / 单位球弧长）
PIX_PER_UNIT = FOCAL_RATIO * PHOTO_SIZE[0] / CAMERA_DISTANCE   # ≈ 163.6
def _lam_px_to_f(lam_px):
    return 2.0 * np.pi * PIX_PER_UNIT / np.asarray(lam_px, dtype=np.float64)

_TEX_MID  = dict(n=64, lam=(3.0, 12.0))   # 中高频：决定关键点密度
_TEX_LOW  = dict(n=8,  lam=(12.0, 60.0))  # 低频：区域可区分性
_TEX_TILT = 0.5                           # 幅度-频率指数 a ∝ f^-tilt
```

幅度按 <code>a ∝ f^-0.5</code> 加权使各频段梯度能量大体相当；方向在三维空间随机保证图案各向同性、无极点与接缝伪影；随机相位保证非周期、无自相似。同时移除造成歧义的周期方格项。

6.3 方案对照实验

同一组 16 视角、同一套确定性 COLMAP 参数下的客观对照（脚本 <code>output/_t_tex.py</code>）：

纹理方案	关键点/图	有匹配像对	内点/对	注册
旧 θ/φ 正弦叠加（base）	403	45 / 120	63	**3 / 16**
随机平面波 wave	2907	75 / 120	366	16 / 16
随机平面波 wave_hi（采用）	**4248**	**76 / 120**	**522**	**16 / 16**
wave_t1（偏粗尺度）	2386	78 / 120	296	16 / 16
wave_lat（加周期方格）	2921	34 / 120	363	更差

> **结论 4**：采用 <code>wave_hi</code>（λ 3~12 px 主频段 + 12~60 px 低频段，共 72 个分量）。关键点密度提升 **10.5 倍**；内点密度 63 → 522/像对（**8.3 倍**），为 SfM 留出充足裕度。
> <code>wave_lat</code> 的结果同时印证了「周期方格破坏匹配」的判断，故不采用。频率也**不可无限推高**：λ 下限压到 3 px 已可用，再细会因混叠使描述子跨视角不可复现。

6.4 渲染性能一并优化

纹理在 **1× 分辨率求值后广播**到 2× 超采样网格。依据是纹理本身带限（最小特征周期约 3 px），2×2 超采样块内取值几乎相同，广播与逐点求值等价，但纹理计算量降到 **1/4**（合成 16 视角由约 90 s 降到约 25 s）。早期尝试的「高斯斑点场」方案因需构建 <code>(N_blobs, N_pixels)</code> 距离矩阵、单图耗时约 85 s 且峰值内存 3.3 GB，已弃用。

七、验证证据

7.1 端到端复现性验收

真实 demo 生成路径（含 JPEG 存储、25°/50° 仰角）× 2 轮，逐项比对：

```
[第1轮] 照片=16 注册=16/16 相机=1 位姿sha1=e535bb60a41b
        DB: 关键点=63140 kp_sha1=e52fcf6ef0f1  内点=39411 gi_sha1=e4049ce91c1d
[第2轮] 照片=16 注册=16/16 相机=1 位姿sha1=e535bb60a41b
        DB: 关键点=63140 kp_sha1=e52fcf6ef0f1  内点=39411 gi_sha1=e4049ce91c1d

--- 验收 ---
  两轮注册数一致: 是       注册率 100%: 是
  DB 比特级一致 : 是       相机中心一致: 是
```

> **验收通过**：database.db 比特级一致、位姿逐位一致、注册率稳定 100%。即论文实验在相同输入下**可完全复现。**

7.2 一键演示全流程

检查项	修复前	修复后
S1 注册	2 / 16（读到碎片模型）	**16 / 16**
M5 纹理覆盖率	16.8%	**29.0%**
M5 纹理锐度	3780.85	1816.96
纹理图非空占比	18.5%	**37.0%**
纹理图唯一色数	23164	13571（且非空占比翻倍）
报表产物	E601 崩溃	**CSV + PNG + HTML 三件套**
批处理退出码	1	**0**（total=1 done=1 failed=0）

注：锐度数值下降是**预期现象**——旧纹理高频成分集中（更「锐」）但匹配失败、模型仅有 2 个相机；新纹理在更多相机下取得更高的有效覆盖，锐度指标应在「覆盖充分」的前提下横向比较。

7.3 打包版实机验证

```
> H3D.exe --batch <config.yaml>       # 打包版（console=False，pyi_dist_v3）
EXIT=0   用时 1m17s

[WARNING] S1 COLMAP 产出 2 个子重建（图像数 [16, 2]）——取最大者 sparse/1
[INFO]    S1 子重建归一：sparse/1 -> sparse/0
[INFO]    S1 完成：demo_relic（16/16 张注册，1 相机，49.7s）
[INFO]    报表生成完成：3 个文件 -> report

run.log 与 batch_gui.log 行数一致（57 行）——_Tee 双写正常
```

7.4 回归测试

测试文件	新增内容	结果
<code>tests/test_runner/test_sfm_solve.py</code>	<code>TestDeterministicRepro</code>（7）、<code>TestIncrementalMetaNotOverwritten</code>（2）、<code>TestMatcherSubcommandAlias</code>（2）、<code>TestLargestSubmodelSelection</code>（7）、<code>TestPreparePhotosCache</code>（4）	38 passed
<code>tests/test_gui.py</code>	<code>TestDemoTextureSiftFriendly</code>（3）：极值密度、全视角可用性、带限无混叠	6 passed
全量	402 项（原 377）	**402 passed**

关键回归测试的**断言设计**：不以「函数不报错」为目标，而是直接锁定行为本质——如「两轮 database.db 逻辑签名必须相同」「mapper 产出 0(2 张)/1(16 张) 时 sparse/0 最终必须是 16 张」「每个示例视角的局部极值数须 > 1200」。这样同类缺陷再次引入时会被立即拦截。

八、交付清单与下一步

8.1 本批交付

类别	内容
修复源码	<code>sfm_solve.py</code>（子模型择优归一 / 确定性线程 / matcher 别名 / inc_meta / 增量目录）、<code>core/config.py</code>（新增 <code>deterministic</code>、<code>num_threads</code>）、<code>gui/demo.py</code>（纹理重构 + 渲染提速）、<code>gui/wizard.py</code>、<code>gui/launcher.py</code>、<code>gui/runner.py</code>、<code>report/plot_gen.py</code>、<code>report/__init__.py</code>、<code>cli.py</code>、<code>core/proc.py</code>（新增）
新增配置	<code>sfm_solve.deterministic</code>（默认 True，论文可复现）、<code>sfm_solve.num_threads</code>（0=自动推导，>0 显式覆盖）
测试	新增 25 项回归，全量 402 passed
诊断脚本	<code>output/_db_sig.py</code>（DB 逻辑签名比对）、<code>output/_t_tex.py</code>（纹理方案对照）、<code>output/_t_e2e.py</code>（端到端两轮验收）
打包	<code>H3D.exe</code>（237.7 MB）重打包；已修正 <code>h3d_gui.spec</code> 中错误的隐藏导入 <code>h3d.report.html</code> → <code>h3d.report.html_export</code>（消除每次打包的 ERROR 行）；已部署至 <code>D:/h3d_relic_test/pyi_dist_v3/</code> 并完成 <code>--batch</code> 冒烟测试（EXIT=0，S1 16/16，报表三件套，<code>batch_gui.log</code> 与 stdout 内容一致）

8.2 确定性模式的使用权衡

<code>deterministic=True</code>（默认）时特征提取、匹配、mapper 三阶段均单线程，换取比特级可复现；代价是**大图集时特征提取与匹配耗时上升**。若某次运行只追求速度、不要求复现（如交互式试跑），可置 <code>deterministic: false</code> 恢复自动多线程。

```
sfm_solve:
  deterministic: true     # 论文实验建议保持 true
  num_threads: 0          # 0 = 按 deterministic 推导(1 或 -1)
  # 追求速度时可:
  # deterministic: false   # -> 特征提取/匹配 -1(全核)、mapper 4 线程
```

8.3 下一步建议

优先级	事项	说明
高	实机真实照片校验	本批所有验证基于合成示例数据。真实文物照片（纹理真实、光照不均、存在遮挡）下的表现需实测；重点观察 S1 注册率与子重建数量

优先级	事项	说明
高	将确定性开关写入论文实验脚本	论文所有对比实验须在 deterministic=true 下运行，并在正文注明参数
中	M4 纹理质量指标复核	纹理覆盖率 29.0% 对该规模示例仍偏低，建议核查 UV 图集利用率与Z-buffer可见性剔除阈值
低	示例纹理观感优化（可选）	现纹理为各向同性带限随机场，视觉上偏「颗粒感」——这是换取 SIFT 关键点密度的直接结果（内点密度 522/像对）。若用于对外演示，可在保持 3~12 px 主频段的前提下适度加强 12~60 px 低频分量权重，使形体起伏更可辨；此项不影响功能，属观感考量

关于「2 张碎片子重建」的澄清：示例项目至今仍会产出一个 2 张的子重建，这**不是**数据连通性缺陷——主重建本身已覆盖全部 16 张照片，该碎片是 COLMAP 首次种子对扩展失败后丢弃的尝试，其图像随后由主重建完整注册（因此两者图像集合存在重叠）。这是 COLMAP 的**固有行为**，正确应对方式就是本批实现的「取图像最多的子重建」，无需修改示例数据。

本批一句话总结：两个表观 BUG 的表层修复只占本次工作的一小部分；真正的价值在于顺着「注册数波动」这条线索，排除了随机种子与线程两个常见误解，最终定位到 **COLMAP 多子重建时误取碎片模型**这一模型选择缺陷，并连带清理了两处静默失效/指标失真问题，使示例全流程从「偶发可用」变为**可复现、可验证、有充足裕度。**